



Real-Time Systems

Schedule search

Student :
Branjauneau Ethan

Year : Aéro4

Tuesday, April 28, 2026
Year 2025-2026

Introduction

The scheduling of tasks in real-time systems requires rigorous temporal validation to ensure that all critical deadlines are strictly met. The objective of this assignment is to analyze the schedulability of a periodic task set consisting of seven tasks and to synthesize an optimal non-preemptive schedule.

This report is structured in two main phases. First, we perform a theoretical and empirical analysis to determine the Worst-Case Execution Time (WCET) of task τ_1 (a large product computation), ensuring the overall processor utilization remains within feasible bounds. Second, we design an optimization algorithm at the job level over the system's hyperperiod. The goal of this algorithm is to minimize the total waiting time of all jobs under two specific scenarios: a strict scheduling constraint where no deadlines are missed, and a relaxed scenario where task τ_5 is explicitly allowed to miss its deadline.

1	Task Set Schedulability	3
1.1	Theoretical WCET for Task 1	3
1.2	Empirical Analysis of Task τ_1	4
2	Job Scheduling and Optimization	5
2.1	Assumptions and Hyperperiod	5
2.2	Algorithm Selection and Computational Complexity	5
2.3	Scenario 1: Strict Schedulability (No Deadlines Missed)	5
2.4	Scenario 2: Relaxed Constraints for τ_5	6
	Conclusion	7

1 Task Set Schedulability

The objective of this section is to determine the theoretical schedulability of the real-time system and define the maximum allowable Worst-Case Execution Time (WCET) for task τ_1 . The system consists of seven periodic tasks. While tasks τ_2 through τ_7 have fixed parameters, τ_1 performs a large product computation whose execution time C_1 must be bounded to ensure system feasibility.

1.1 Theoretical WCET for Task 1

The task set parameters, as defined in the assignment, are summarized in the table below:

Task	Execution Time (C_i)	Period (T_i)	Utilization (U_i)
τ_1	C_1	10	$C_1/10$
τ_2	3	10	0.3000
τ_3	2	20	0.1000
τ_4	2	20	0.1000
τ_5	2	40	0.0500
τ_6	2	40	0.0500
τ_7	3	80	0.0375

Table 1: Real-time task set parameters.

The total processor utilization U must satisfy the condition $U \leq 1$ for the system to be theoretically schedulable. We first calculate the cumulative utilization of the fixed tasks (τ_2 to τ_7):

$$U_{fixed} = \sum_{i=2}^7 \frac{C_i}{T_i} = 0.3 + 0.1 + 0.1 + 0.05 + 0.05 + 0.0375 = 0.6375 \quad (1)$$

To maintain schedulability, the addition of τ_1 must not cause the total utilization to exceed 100%:

$$U_{total} = \frac{C_1}{10} + 0.6375 \leq 1 \quad (2)$$

Solving for C_1 :

- $\frac{C_1}{10} \leq 1 - 0.6375$
- $\frac{C_1}{10} \leq 0.3625$
- $C_1 \leq 3.625$ ms

The system is theoretically schedulable as long as the WCET of τ_1 does not exceed **3.625 ms**. In the following sections, we use the results from our C implementation to verify that

the measured execution time of the large product computation, including a 20% safety margin, remains within this critical bound.

1.2 Empirical Analysis of Task τ_1

To determine the actual computation time of task τ_1 , we measured the execution times of the large product computation over multiple iterations. The statistical results of these measurements are summarized below:

- **Minimum:** 0.061218 ms
- **1st Quartile (Q1):** 0.062017 ms
- **Median (Q2):** 0.063183 ms
- **3rd Quartile (Q3):** 0.089640 ms
- **Maximum:** 2.126709 ms

While the median and quartiles show that the task typically executes very quickly (under 0.09 ms), the maximum execution time reaches 2.126709 ms. This peak is expected in real-world measurements due to operating system overhead, such as context switching or cache misses.

To ensure the reliability of our real-time system and to account for unpredictable delays, we apply a 20% safety margin to the maximum observed execution time. Consequently, the measured Worst-Case Execution Time (WCET) for task τ_1 is calculated as:

$$C_1 = 2.126709 \times 1.20 = 2.552051 \text{ ms}$$

Since this calculated C_1 (2.552051 ms) is strictly less than our theoretical upper bound of 3.625 ms, the total processor utilization remains safely under 100% ($U_{total} \approx 0.893$). Thus, the task set is confirmed to be theoretically schedulable.

2 Job Scheduling and Optimization

2.1 Assumptions and Hyperperiod

To schedule this periodic task set at the job level, we must evaluate it over its hyperperiod, which is the Least Common Multiple (LCM) of all task periods.

$$LCM(10, 10, 20, 20, 40, 40, 80) = 80 \text{ ms}$$

Over this 80 ms interval, the system generates a total of $N = 29$ jobs. The schedule generated within this hyperperiod will repeat cyclically ad infinitum.

Note on Processor Idle Time: The assignment suggests verifying if minimizing the total waiting time implies maximizing the total processor idle time. However, over a fixed hyperperiod, the total execution time is a constant defined by the sum of all job execution times:

$$C_{total} = \sum_{k=1}^N C_k = 71.416 \text{ ms}$$

Consequently, the total processor idle time over the hyperperiod is strictly constant:

$$Idle_{total} = Hyperperiod - C_{total} = 80 - 71.416 = 8.584 \text{ ms}$$

Changing the execution order to minimize waiting time does not change the *total* idle time; it only changes *when* the processor is idle. The objective of our algorithm is therefore purely to minimize the latency (waiting time) of individual jobs without missing deadlines.

2.2 Algorithm Selection and Computational Complexity

The problem of finding an optimal non-preemptive schedule that minimizes total waiting time subject to strict deadline constraints is known to be NP-Hard. A naive exhaustive search (like a pure Branch and Bound without heuristic pruning) evaluating every permutation of N jobs would have a worst-case computational complexity of $\mathcal{O}(N!)$. For our system of 29 jobs, 29! operations are computationally intractable.

To solve this efficiently, we implemented a **Constraint Programming (CP)** model using the Google OR-Tools solver. The CP solver uses advanced constraint propagation and domain filtering to drastically prune the search space, allowing it to find the mathematical global optimum in a fraction of a second.

2.3 Scenario 1: Strict Schedulability (No Deadlines Missed)

In this first scenario, strict deadline constraints were applied to all 29 jobs. The solver successfully found a feasible, optimal schedule where **zero deadlines are missed**. The minimized total

waiting time across all jobs is **149.216 ms**. The response time of each job was calculated using:
 $R = \text{End_Time} - \text{Arrival_Time}$.

Job ID	Arrival	Start	End	Wait Time	Resp. Time	Deadline	Met?
T1_job1	0.0	4.000	6.552	4.000	6.552	10.0	YES
T1_job2	10.0	13.552	16.104	3.552	6.104	20.0	YES
T1_job3	20.0	24.104	26.656	4.104	6.656	30.0	YES
T1_job4	30.0	31.656	34.208	1.656	4.208	40.0	YES
T1_job5	40.0	44.000	46.552	4.000	6.552	50.0	YES
T1_job6	50.0	53.552	56.104	3.552	6.104	60.0	YES
T1_job7	60.0	64.000	66.552	4.000	6.552	70.0	YES
T1_job8	70.0	70.000	72.552	0.000	2.552	80.0	YES
T2_job1	0.0	6.552	9.552	6.552	9.552	10.0	YES
T2_job2	10.0	16.104	19.104	6.104	9.104	20.0	YES
...	...	...	...	...	...	...	...
T3_job1	0.0	0.000	2.000	0.000	2.000	20.0	YES
T4_job1	0.0	2.000	4.000	2.000	4.000	20.0	YES
T5_job1	0.0	9.552	11.552	9.552	11.552	40.0	YES
T6_job1	0.0	11.552	13.552	11.552	13.552	40.0	YES
T7_job1	0.0	19.104	22.104	19.104	22.104	80.0	YES

Table 2: Excerpt of the strict optimal schedule (Total Wait = 149.216 ms)

(Note: The full 29-job schedule is verified in the source code repository attached to this report.)

2.4 Scenario 2: Relaxed Constraints for τ_5

In this second scenario, the deadline constraint for task τ_5 was explicitly removed, allowing its jobs to potentially finish after their respective deadlines to see if it would yield a better overall waiting time for the system.

Interestingly, the solver returned the exact same optimal waiting time of **149.216 ms**, and the resulting schedule still met all deadlines, including those of τ_5 . This demonstrates that the optimal packing of jobs achieved in Scenario 1 is the absolute global minimum. Relaxing the constraint on τ_5 does not provide any additional benefit to the overall system latency, proving the high schedulability and robustness of the task set.

Conclusion

In this project, we successfully demonstrated the schedulability of the proposed real-time task set. Through empirical measurement and the application of a 20% safety margin, the WCET of task τ_1 was bounded at 2.552 ms. This maintained the total processor utilization at a safe 89.3%, well below the theoretical threshold.

For the scheduling phase, the use of a Constraint Programming model (via Google OR-Tools) proved highly effective in navigating the NP-Hard complexity of non-preemptive scheduling. We achieved a strictly feasible optimal schedule with a minimized total waiting time of 149.216 ms over the 80 ms hyperperiod. As demonstrated, while the algorithm optimizes individual job latencies, the total processor idle time remains mathematically constant (8.584 ms) based on the total execution workload, refuting the assumption that minimizing wait time maximizes overall idle time.

Finally, relaxing the deadline constraint for task τ_5 did not yield a lower waiting time, as the strict scenario had already reached the absolute mathematical global optimum. This confirms both the high robustness of the given task set and the superiority of an exact optimization approach over standard heuristics. The complete source code reproducing these empirical measurements and schedules is available in the associated Git repository.